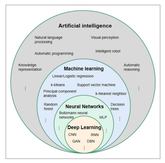
Study on Some Machine Learning Algorithms used in Production Data Analysis

Ramu Battu, Joel A. Espinoza, David Mercado, Harshdeep Singh, Sarbrinder P. Singh, Nitaigour P. Mahalik,
Department of Industrial Technology, Jordan College of Agricultural Science and Technology, California State University, Fresno, California, USA

Abstract

Production Data Analysis (PDA) entails classification, regression, and dimensionality reduction tasks using various tools, techniques, methods, and approaches. Due to the advent of sophisticated computational paradigms, this study investigates the important Machine Learning (ML) algorithms used for production data analysis. Figure 1 shows the types of data the production and manufacturing system deal with. This research highlights several important algorithms that are not only aligned with Artificial Intelligence (AI) principles but also demonstrate coding efficiency and computationally validated flow structures. The presentation starts with conventional statistical tools, and techniques to reinforcement learning. In particular descriptive and inferential statistics, sampling, Regression Analysis, Logistic Regression, Time Series Analysis, Support Vector Machines, Principal Component Analysis (PCA), Decision Trees, AdaBoost, Random Forest, Gradient Boost, kNN, k-Means, and Q-Learning.

INTRODUCTION



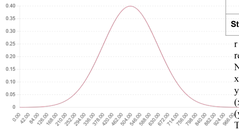
Learning Type	Nature of Data	Learning Objective
Supervised Learning	Uses labeled data (input-output pairs known)	Predict outcomes (classification or regression) based on examples
Unsupervised Learning	Uses unlabeled data (only inputs, no known outputs)	Discover hidden patterns or structure in the data (clustering, association)
Reinforcement Learning	Learns from interaction with environment using rewards or penalties	Learn optimal actions or policies that maximize cumulative reward through trial and error.

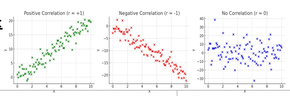
DESCRIPTIVE & INFERENTIAL STATISTICS

Descriptive & Inferential Statistics:

- Deals with entirety of datasets.
- Summarizes, visualizes, describes.
- Key tools:
- **Central Tendency:** mean, median, mode
- **Dispersion:** variance, standard deviation
- Other Tools:Covariance and Correlation

- ***Inferential:***
- Takes a sample of entire dataset.
- Estimates, tests hypotheses, makes predictions using different methods.





Pearson Correlation Coefficient:	$r=\sum_i^n N*(xi-x^-)(yi-y^-)/[SQRT(\sum_i^n N(xi-x^-)^2)][SQRT(\sum_i^n N (yi-y^-)^2)]$
Population Variance	$\sigma^2=(1/N)\sum N(xi-\mu)^2$
Covariance	$(X,Y)=1/N=\sum(xi-x^-)(yi-y^-)$
Standard Deviation	σ=SQRT(1/N*$\sum$(xi-μ)^2)

r = Pearson correlation coefficient (measures linear relationship between X and Y)
N= total number of paired observations
x^- = mean (average) of variable X
y^- = mean (average) of variable Y
(xi−x^-) = deviation of each X value from its mean
(yi−y^-) = deviation of each Y value from its mean
The denominator [SQRT(∑i^N(xi−x^-)^2)][SQRT(∑i^N (yi−y^-)^2)]= product of the standard deviations of X and Y
N = total number of observations in the population
xi = value of the i-th observation of variable X
yi = value of the i-th observation of variable Y
μX = population mean of variable X
μY = population mean of variable Y
(xi−μX)(yi−μY)= product of deviations of each observation from their respective means

Correlation:

- Measures relationship strength.
- Normalized, always between -1 and +1.
- **EX:** Study Hours = [1,2,3,4,5], Exam Scores = [50,60,65,70,80]

1. Compute means of X and Y
2. Compute deviations from means for X and Y
3. Multiply deviations pairwise and sum
4. Divide by product of standard deviations of X and Y
- Result: r = 0.97 (Strong Positive Correlation)

Covariance:

- Measures direction of joint variability, or how much the datasets change together by.
- Value depends on scale of variables.
- Cov(X,Y) > 0 (both increase together).
- **EX:** X = [2, 4, 6], Y = [1, 3, 5]

1. Mean of X = 4, Mean of Y = 3
2. Deviations X = [-2,0,2], Y = [-2,0,2]
3. Multiply deviations = [4,0,4]
4. Covariance = (4+0+4)/(3-1) = 4

μ = population mean
(xi−μ)^2 = squared deviation of each observation from the mean

REGRESSION: LINEAR, NONLINEAR, LOGISTIC

Developing and Evaluating a Model:

1. **Define the Problem to Predict Outcomes For**
2. **Collect Production Data**
   a. (i.e. Pressure, Humidity, Defect Rate)
3. **Specify Model**
4. **Estimate Coefficients (using software)**
5. **Predict and Execute Model**
6. **Evaluation and Optimization of the Model - i.e. R², AIC**

Linear Model Equation	$Y = \beta_0 + \beta_1 X + \varepsilon$
Multi Linear Equation	$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \ldots + \varepsilon$
Polynomial Regression	$Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \varepsilon$
Logistic Regression	$\log(p/(1-p)) = \beta_0 + \beta_1 X$
Variance Inflation Factor:	$VIF=1/(1-R^2)$
Adjusted coefficient of determination:	$R(adj.)^2=1-[(1-R^2)(n-1)/(n-k-1)]$

Evaluation Techniques:

- Analyze residuals (error) to ensure they are uniformly randomly scattered. Constant variance is called **HOMOSCEDASTICITY**.
- Check **R^2:** a measure of goodness of fit for more than one predictor.
  - Adjusted just subtracts from 1 for better representation.
- Check **p-values** for coefficients to ensure each ind variable significantly affect defect rate.

Terminology

Y (Dependent Variable)
This is what we are trying to predict or explain. **Ex**: Number of units produced per day, production time per batch, defect rate, or machine downtime.

X (Independent Variable)
This is the factor that influences Y. **Ex**: Number of workers on the line, hours of machine operation, raw material quality score, or temperature in the production area.

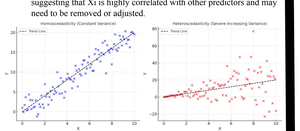
β_0 (Intercept)
This is the value of Y when X = 0. **Ex**: If X = hours of machine operation per day, β_0 is the baseline production output if no machines run.

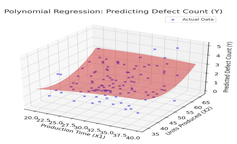
β_1 (Slope Coefficient)
This measures how much Y changes for a one-unit increase in X. **Ex:** If X = number of workers, β_1 tells how many additional units are produced for each extra worker.

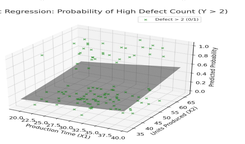
ε (Error Term / Residual)
This captures all other factors affecting Y that are not included in X.

Refining and Classifying the Model:

- Split production data into training/testing sets and cross-validate.
- Check predictive accuracy and plot errors for visualization.
- Compare predicted vs observed values
- Using **VIF** - Test for **Multicollinearity**: occurs when predictors highly correlated and β becomes sensitive to small changes in data.
- Interpretation:
- • VIF=1: No correlation between Xi and other predictors.
- • VIF>10 (or sometimes >5): Indicates high multicollinearity, suggesting that Xi is highly correlated with other predictors and may need to be removed or adjusted.







AR, MA, ARMA

Autoregressive - A model

(Not a statistical property like autocorrelation)

- Autoregressive model uses those correlations to build a predictive equation.
- AR is part of a modeling framework

$X_t = c + \varphi_1 X_{t-1} + \varphi_2 X_{t-2} + \cdots + \varphi_p X_{t-p} + \varepsilon_t$

- φi are model parameters that quantify how much each lag affects the current value.
- If daily production depends on yesterday's output (with persistence 0.8):

$X_t = 20 + 0.8X_{t-1} + \varepsilon_t$

Moving Average (MA)

- Two concepts:
  - Smoothing Moving Average (Short-Term – for trend visualization)
  - Moving Average Model (MA)
- For a window of size m:

$MA = (X_t + X_{t-1} + X_{t-2} + \cdots + X_{t-(m-1)}) / m$

- An MA(q) model expresses the current value as a linear combination of past random shocks (errors).

$X_t = \mu + \varepsilon_t + \theta_1\varepsilon_{t-1} + \theta_2\varepsilon_{t-2} + \cdots + \theta_q\varepsilon_{t-q}$

X_t: current production
ε_t: random noise (e.g., unexpected machine downtime, raw material shortage)
θ_i: coefficients showing how past errors affect current production
q: order of the MA model (how many past shocks matter)

$X_t = 100 + \varepsilon_t + 0.5\varepsilon_{t-1}$

ARMA Model Building

- It is a fundamental statistical model used for time series analysis and forecasting. Two key ideas:
  - Auto-Regressive (AR) part – uses past values of the series to predict the current value.
  - Moving Average (MA) part – uses past forecast errors (shocks) to improve predictions.
- The model assumes that the data is stationary (statistical properties mean and variance do not change over time).

$Y_t = c + \Sigma(\varphi_i Y_{t-i}) + \Sigma(\theta_i\varepsilon_{t-i}) + \varepsilon_t$

Where:
Y_t = value of the time series at time t
c = constant term
φ_i = coefficients for the autoregressive (AR) terms
θ_i = coefficients for the moving average (MA) terms
ε_t = random white noise (error term)
p = order of AR part
q = order of MA part

ARMA Interpretation: AR(p) captures how the current value depends on its previous p values.

- MA(q) captures how the current value depends on previous q random shocks (errors).
- ARMA (p, q) combines both memory from past observations and correction from past prediction errors.

Fitting an ARMA(2,1) Model

$Y_t = c + \varphi_1 Y_{t-1} + \varphi_2 Y_{t-2} + \theta_1\varepsilon_{t-1} + \varepsilon_t$

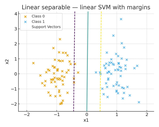
Assume after estimation we get φ_1=0.6, φ_2=0.3, θ_1=0.5

Then, for day 11:
$\hat{Y}_{11} = 1.2 + 0.6Y_{10} + 0.3Y_9 + 0.5\varepsilon_{10}$
If Y_{10}=114, Y_9=115, and ε_{10}=-0.8:
$\hat{Y}_{11} = 1.2 + (0.6)(114) + (0.3)(115) + (0.5)(-0.8)$

SUPPORT VECTOR MACHINE (SVM)

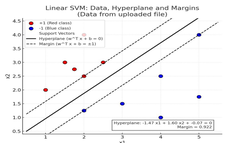
Support Vector Machines (SVM)

- Finds the optimal separating hyperplane between classes
- Effective in high-dimensional spaces
- Example: Classifying data
- Can use kernel functions for non-linear classification



EX: There are 10 data points used for the Support Vector Machine (SVM) computation. Each data point has two features — X_1 and X_2, along with a class label.

1. $X_1 = 1$, $X_2 = 2$ → Class = +1 (Red)
2. $X_1 = 1.5$, $X_2 = 2.1$ → Class = +1 (Red)
3. $X_1 = 1.75$, $X_2 = 2.5$ → Class = +1 (Red)
4. $X_1 = 2$, $X_2 = 2.5$ → Class = +1 (Red)
5. $X_1 = 2.5$, $X_2 = 3.1$ → Class = +1 (Red)
6. $X_1 = 3$, $X_2 = 1.25$ → Class = −1 (Blue)
7. $X_1 = 3.5$, $X_2 = 1.5$ → Class = −1 (Blue)
8. $X_1 = 4$, $X_2 = 1.75$ → Class = −1 (Blue)
9. $X_1 = 4.5$, $X_2 = 1.75$ → Class = −1 (Blue)
10. $X_1 = 5$, $X_2 = 1.8$ → Class = −1 (Blue)



Dual Coefficients (α values)

dual_coef = [-1.86628985, -0.4883726, 2.35512664]

These represent the Lagrange multipliers for the support vectors used in training the linear SVM.

Compute Weight Vector (w)

$w = \Sigma (\alpha_i y_i x_i)$

Substituting values:

w = (2.35512664)(+1)(2, 2.5) + (1.86628985)(+1)(2, 2.5) + (0.4883726)(−1)(3.5, 1.5)

Result: w = (−1.46648281, 1.59963229)

Compute Bias Term (b)

$b = \text{average}(y_i - (w \cdot x_i))$

For support vectors, calculate individual b values and take their mean:

b_1 = -0.0663, b_2 = -0.0663, b_3 = -0.0663

Average bias: b = −0.06626831

Norm of w and Margin

‖w‖ = √((-1.466)^2 + (1.599)^2) = 2.170141142

Margin = 2 / ‖w‖ = 0.92161052

Final Numeric Summary:w: (−1.46648281, 1.59963229)b: −0.06626831‖w‖: 2.170141142Margin: 0.9216105

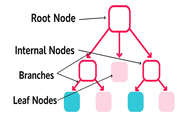
DECISION TREE AND RANDOM FOREST

Decision Trees

- Hierarchical model for Regression and Classification
- Splits data using features to minimize impurity (Gini, Entropy)
- Easy to interpret and visualize
- Example: Predicting machine breakdown cause
- Can **overfit** without pruning or depth limitation

Components of Decision Trees

- Root Node
- Internal Nodes (Child Nodes)
- Branches
- Leaf Nodes



A decision is made at each internal node of a decision tree. These decision nodes represent a test on an attribute/feature, which splits the data into smaller subsets based on the outcome of that test, with each branch representing a possible outcome. The tree starts with a root node, which is the initial decision, and continues to branch out until it reaches the leaf nodes, which represent the final outcome or decision

Decision Tree – Gini Impurity

$G = 1 - \Sigma (p_i)^2$

G_split = (n_L / n) * G_L + (n_R / n) * G_R

Random Forest

- Ensemble of multiple decision trees
- Reduces overfitting and improves accuracy
- Uses bagging (bootstrap aggregation) method
- Example: Predicting product defects in large datasets
- Feature-importance ranking built-in

Random Forest – Feature Importance

$Imp(j) = [\Sigma_{T \in N_T} (j) w_T \Delta i(n_T)] / [\Sigma_{T \in N_T} w_T \Delta i(n_T)]$

where Δi(n) = impurity decrease at node n.

ADABOOST AND GRADIENT BOOST

ADABOOST

- Train weak learners to strong classifiers.
- Reweight misclassified samples to focus on harder points.
- Pros: Simple, strong on tabular data.
- cons: Sensitive to label noise/outliers.
- Knob: Number of estimators.
- Use: Classification (Spam , Face detection)

Math:

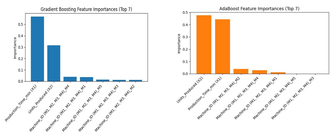
$\alpha_t = ½ \ln((1 - \varepsilon_t)/\varepsilon_t)$
$w_i \leftarrow w_i e^{-\alpha_t y_i h_t(x_i)}$

Gradient Boost

- Sequentially add models to reduce prediction errors.
- Each new tree fits the residuals (negative gradients) of loss.
- Pros: Very accurate, flexible, works for regression and classification.
- Cons: Slower to train, can overfit without tuning.
- Knob: Learning rate, number of trees, tree depth.
- Use: Regression, ranking, structured/tabular data.

Math:

$\hat{y}_{t+1} = \hat{y}_t + \eta f_t(x)$



Results: **AdaBoost** achieved around 18% accuracy, with limited performance as it struggled with multi-class noise and small sample imbalance. While **Gradient Boosting** reached around 29% accuracy, outperforming AdaBoost by learning complex patterns and reducing classification error across most classes

K-NEAREST NEIGHBOR (kNN) and K-MEAN

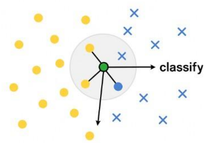
K-Nearest Neighbor (kNN)

• A distance based classification method that assigns a new observation to the majority class of its k nearest neighbors.

• Useful in production data for predicting defect vs. non-defect labels by comparing a product's features (temperature, pressure, quality) to similar historical samples.

• Pros: Simple, effective on small datasets.
• Cons: Slow on large data; results depend on the choice of k.

K-Nearest Neighbor (kNN)



K-Means Clustering

• An unsupervised algorithm that groups data into K clusters by minimizing the distance between points and their cluster centroid.

• Objective: minimize within cluster variability
• Steps: Initialize K centroids → assign points → recompute centroids → repeat.

• Used in production settings to detect major differences, segment defect patterns, identify machine operating states, and cluster similar process conditions.

• Pros: Fast, scalable
• Cons: Requires selecting K; sensitive to outliers

ARTIFICIAL NEURAL NETWORK (ANN) AND CNN

ANN

Artificial Neural Networks (ANN) consists of nodes and connections between nodes. The nodes along each connection represents parameter values. Parameter values are optimized using a method called back-propagation. The weight(parameter value) is multiplied by input value and biases (determined by Gradient descent) are added together to fit the data. Using weights and biases flips and stretches the activation functions into new shapes. There are three activation functions,they determine corresponding Y- values. This is how we move from one node to another.

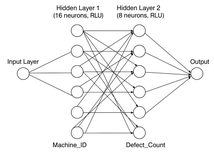


Fig1: ANN architecture diagram using RELU as activation function

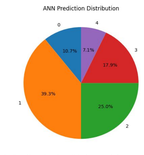


Fig3: Predicted value (%) by model across all test samples.

1) Softplus: $F(x) = Log(1 + e^x)$
2) Sigmoid: $f(z) = \frac{e^z}{e^z+1}$
3) ReLU : $f(z) = \max(0,z)$
4) TanH : $f(z) = \tanh(z)$
5) Backpropagation
a) Output Layer: δ(L)=∇aL.f′(z(L))
b) Hidden Layer error:
δ(l)=((W(l+1))Tδ(l+1)). f′(z(l))
Where
L= Output Layer
δ= Error Signal

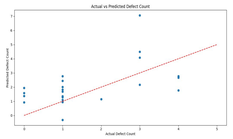


Fig2: Actual defects and predicted defects by ANN model

a) **MAE was 1.22 and R^2 was -0.23 which indicates that learned mapping between input and defects is not reliable owing to noise in data.**

b) **b) ANN model tried to learn about certain machine, if production rate is high and number of units produced are low or high, how does that affect defect count.**

Q-LEARNING

Q-learning is a **model-free reinforcement learning** method used to learn the quality of actions without knowing how the environment works. The quality function, Q(s,a), represents how good it is to take action a in state s. Although Q-values are formally defined using Markov Decision Processes (MDPs), which assume probabilistic state transitions, real systems often do not provide these probabilities or reward structures beforehand. Because the model is unknown, learning must occur through experience. Two major approaches are used: **Monte Carlo** methods, which attribute the final reward to all past actions in an episode, and **Temporal Difference (TD)** learning, which updates values based on the most relevant past step. TD(0) looks only one step ahead, while **TD(λ)** blends multiple time steps using weights between 0 and 1, combining short-term and long-term information. **Q-learning** applies TD learning directly to the Q-function, enabling learning of optimal actions without a model.

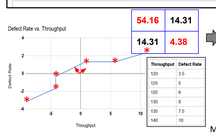
Mathematical Foundation	
1) $V(s)=Q(s,a) = E[R(s',s,a) + \gamma v(s')]$	**State value function using expectation**
2) $\sum P(s'\|s,a) [R(s',s,a) + \gamma v(s')]$	**When next stage is probabilistic**
3) $V^{new}(S_k) = V^{old}(S_k) + \alpha[r_k + \gamma V^{old}(S_{k+1}) - V^{old}(S_k)]$	**TD Value**
4) $Q^{new}(S_k,a_k) = Q^{old}(S_k,a_k) + \alpha (r_k+\gamma \max_a Q(S_{k+1}, a) - Q^{old}(S_k,a_k)$	**Q- Learning Equation**

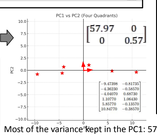
Symbol	Meaning	Symbol	Meaning
S_k	Current State	γ	Discount Factor
S_{k+1}	Next State	α	Learning Rate
a_k	Action taken	$(r_k + \gamma V(s_{k+1})$	Update target
r_k	Reward	$V(s)= \max_a Q(s,a)$	Best Q value in next state
Q^{old}	Current Q value	Q^{new}	TD Update
$\max_a Q(S_{k+1}, a)$	Best Q value	V^{new}	Future Value

PRINCIPAL COMPONENT ANALYSIS (PCA)

PCA flow chart	Subroutine description
$Z = \frac{X-\mu}{\sigma}$	Standardize data
$C = \frac{1}{n-1} Z^T Z$	Compute covariance matrix
$Cv_i = \lambda_i v_i$	Eigen decomposition
Sort λ_i	Rank principal components
$Y = ZV_k$	Transform data into PC space
$\frac{\lambda_i}{\sum \lambda_j}$	Compute variance explained

X	Original data matrix with n samples and p features
n	Number of observations (data points)
p	Number of features (variables)
μ	Mean of each feature
σ	Standard deviation of each feature
Z	Standardized data matrix: $Z = \frac{X-\mu}{\sigma}$
C	Covariance matrix: $C = \frac{1}{n-1} Z^T Z$
c_{ij}	Covariance between feature i and feature j
v_i	Eigenvector representing a principal component direction
λ_i	Eigenvalue representing variance captured by component i
$\lambda_1 \geq \lambda_2 \geq \cdots$	Ordered eigenvalues (descending)
V_k	Matrix of top k eigenvectors (principal axes)
Y	Transformed data in the principal component space: $Y = ZV_k$
$\frac{\lambda_i}{\sum \lambda_j}$	Proportion of total variance explained by component i





Most of the variance kept in the PC1: 57.97/(57.97+0.57)x100= 99%

CONCLUSIONS

Machine Learning (ML) has significant applications in Production Data Analysis (PDA) enabling deeper insights and smarter decision-making across complex industrial processes. By integrating descriptive and inferential statistics with supervised, unsupervised, and reinforcement learning tools, techniques, methods, and approaches organizations can model patterns, forecast performance, and optimize operations. Techniques such as regression, AR, MA, and ARMA enhance predictive capability, while PCA, SVM, ANN, decision trees, random forests, Ada-Boost, and kNN strengthen classification and pattern recognition. Unsupervised tools like k-Means reveal hidden structures, and reinforcement methods like Q-Learning support continuous process improvement. These algorithms lay foundation for modern production and manufacturing systems..

REFERENCES

1. Schapire, R. E. (2003). The boosting approach to machine learning: An overview. In *Nonlinear estimation and classification*. Springer New York, 149–171.
2. Jerez, J. M., Molina, I., García-Laencina, P. J., Alba, E., Ribelles, N., Martín, M., & Franco, L.(2010). Missing data imputation using statistical and machine learning methods in a real breast cancer problem. *Artificial intelligence in medicine*, 50 (2), 105–115.
3. Ben-Hur, A., Horn, D., Siegelmann, H. T., & Vapnik, V. (2001). Support vector clustering. *Journal of machine learning research*, 2 (Dec), 125–137.Lee, J., Bagheri, B., & Kao, H.-A. (2015). A cyber-physical systems architecture for Industry 4.0-based manufacturing systems. Manufacturing Letters, 3, 18–23. Elsevier.
4. Kotsiantis, S. (2015). Supervised machine learning: A review of classification techniques. Informatica, 31(3), 249–268. IOS Press.
5. Hinton, G., & Salakhutdinov, R. (2015). Discovering structure in data by unsupervised learning. Nature, 521, 436–444. Nature Publishing Group.
6. Mnih, V., et al. (2015). Human-level control through deep reinforcement learning. Nature, 518, 529–533. Nature Publishing Group.
7. Sakia, R. (2016). The Box-Cox transformation: A review. Journal of the Royal Statistical Society, 33, 169–178. Sage.
8. Wasserstein, R. L., & Lazar, N. A. (2016). The ASA's statement on p-values. The American Statistician, 70(2), 129–133. Taylor & Francis.
9. Chollet, F. (2017). Deep learning with linear and logistic regression models. IEEE Transactions on Neural Networks and Learning Systems, 28(7). 1520–1532. IEEE.
10. Gómez, V., & Maravall, A. (2016). Program TSW: Trend-cycle estimation with moving averages. Journal of Time Series Analysis, 37(5), 649–670. Wiley.
11. Lütkepohl, H. (2017). Autoregressive models. Handbook of Macroeconometrics, Springer, 55–90. Springer.
12. Box, G., Jenkins, G., Reinsel, G., & Ljung, G. (2016). Time series analysis: Forecasting and control (5th ed.). Wiley.
13. Shlens, J. (2016). A tutorial on principal component analysis. arXiv:1404.1100. Springer (widely cited tutorial version).
14. Cao, Y., & Tay, F. (2018). Support vector machine with adaptive parameters for classification. IEEE Transactions on Neural Networks and Learning Systems, 29(1), 40–52. IEEE.
15. LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. Nature, 521, 436–444. Nature Publishing Group.
16. Silver, D., et al. (2017). Mastering the game of Go without human knowledge. Nature, 550, 354–359. Nature Publishing Group.
17. Fernández-Delgado, M., Cernadas, E., & Barro, S. (2015). Do we need hundreds of classifiers to solve real-world problems? Journal of Machine Learning Research, 15, 3133–3181. JMLR.
18. Qi, Y. (2015). Random forest for bioinformatics. In Encyclopedia of Systems Biology (pp. 1483–1485). Springer.
19. Li, Y., & Zhou, Z.-H. (2017). Improving AdaBoost classifier using new weak learner evaluation. Pattern Recognition, 60, 346–356. Elsevier.
20. Peterson, L. (2016). K-nearest neighbor algorithm overview. Machine Learning Review, 5(2), 1–17. Springer.
21. Celebi, M. E., et al. (2016). A comparative study of k-means algorithms. Expert Systems with Applications, 42(4), 1–15. Elsevier.